

CS 473, Spring 2023

Midterm 1 Solutions

1. (a) Answer: $\Theta(n^{3/2})$. This follows from the master theorem, since $\log_{473} 374 < 3/2$.
- (b) False. Karatsuba's algorithm runs in $O(n^{1.59})$ time, but the FFT-based algorithm (by Cooley and Tukey) runs in $O(n \log n)$ time and is faster. (The algorithm by Toom and Cook is also faster.)
- (c) No. Strassen's algorithm uses subtraction (inverse to addition), but the "max" operation does not have an inverse.
- (d) Since $i \leq i' \leq j \leq j'$ and $p_i, p_{i'}, p_j, p_{j'}$ are on the unit circle in ccw order, the line segment $p_i p_j$ and $p_{i'} p_{j'}$ intersect (a picture would help here). Let q be the intersection point. By applying the triangle inequality twice,

$$\begin{aligned}\|p_i - p_{j'}\| &\leq \|p_i - q\| + \|q - p_{j'}\| \\ \|p_{i'} - p_j\| &\leq \|p_{i'} - q\| + \|q - p_j\|.\end{aligned}$$

Adding the two inequalities yields

$$\begin{aligned}\|p_i - p_{j'}\| + \|p_{i'} - p_j\| &\leq (\|p_i - q\| + \|q - p_{j'}\|) + (\|p_{i'} - q\| + \|q - p_j\|) \\ &= \|p_i - p_j\| + \|p_{i'} - p_{j'}\|.\end{aligned}$$

Thus, $d(i, j') + d(i', j) \leq d(i, j) + d(i', j')$.

- (e) There are 3 recursive calls with input size $n/2$. But since space is freed up after each call, the space usage satisfies the recurrence

$$S(n) = S(n/2) + O(n),$$

which solves to $S(n) = O(n)$.

[Alternatively: the space for the recursion stack, at any time, is bounded by $O(n + n/2 + n/4 + \dots) = O(n)$.]

2. (a) *Pseudocode.*

Product($p_1, \dots, p_n$):

1. if $n = 1$ then return p_1
2. $Q = \text{Product}(p_1, \dots, p_{\lfloor n/2 \rfloor})$
3. $R = \text{Product}(p_{\lfloor n/2 \rfloor + 1}, \dots, p_n)$
4. return $Q \cdot R$

In line 4, the product of the two polynomials is computed by using the FFT-based algorithm from class.

Run time analysis. Observe that the polynomials Q and R have degree bounded by $m(n/2) = O(mn)$. So the FFT algorithm invoked by line 4 takes $O((mn) \log(mn))$ time. Let $T(n, m)$ be the run time of $\text{Product}(p_1, \dots, p_n)$ for n polynomials $p_1, \dots, p_n$ of degree at most m . Then $T(n, m)$ satisfies the following recurrence:

$$T(n, m) = \begin{cases} O(m) & \text{if } n = 1 \\ 2T(n/2, m) + O(nm \log(nm)) & \text{else} \end{cases}$$

To upper-bound $T(n, m)$, we can pull out a factor of $m \log(nm)$ and get the standard mergesort recurrence $T(n) = 2T(n/2) + O(n)$, which solves to $O(n \log n)$. This implies that $T(n, m) = O(nm \log(nm) \log n)$.

[Alternative way to solve the recurrence: consider the recursion tree. Each level sums to at most $O(nm \log(nm))$, and there are $O(\log n)$ levels.]

- (b) Let $S = \{a_1, \dots, a_n\}$. For each $i = 1, \dots, n$, define the polynomial $p_i(x) = 1 + x^{a_i}$, which has degree $a_i \leq U$. Now, compute their product $Q(x) = p_1(x)p_2(x) \cdots p_n(x)$ by part (a), with $m = U$. This takes time $O(nU \log(nU) \log n)$.

Now, to decide whether there exists a subset of S summing to W , we just inspect the coefficient of the x^W term and check whether it is positive.

Total time: $O(nU \log(nU) \log n)$ (which is indeed better than $O(nW)$ if U is much smaller than W).

3. Pseudocode for determining whether answer is yes/no.

1. for $i = 1$ to n do
2. for $x = 0$ to U do $C[i, i, x] = \text{true}$ if $x = a_i$, false else
3. for $j = 1$ to n do
4. for $i = j - 1$ to 1 do
5. for $x = 0$ to U do
6. $C[i, j, x] = \bigvee_{k=i}^{j-1} \bigvee_{y=x}^U (C[i, k, y] \wedge C[k+1, j, y-x])$
 $\quad \vee \bigvee_{k=i}^{j-1} \bigvee_{y=0}^{U-x} (C[i, k, y] \wedge C[k+1, j, y+x])$
7. if $C[i, j, x]$ is true then let $k\text{-pred}[i, j, x]$ and $y\text{-pred}[i, j, x]$ be some k and y
8. s.t. $(y \geq x \text{ and } C[i, k, y] \wedge C[k+1, j, y-x])$ or
 $(y \leq U-x \text{ and } C[i, k, y] \wedge C[k+1, j, y+x])$
9. return $C[1, n, b]$

(Lines 7–8 are for helping out the later pseudocode to output the expression.)

The above uses the evaluation order: increasing j , and for the same j , decreasing i . (There are other alternatives, e.g., increasing $j - i$ would also work.)

Pseudocode for outputting the expression.

OUTPUT-EXPR(i, j, x) :

1. if $i = j$ then output a_i and return

2. $k = \text{k-pred}[i, j, x], y = \text{y-pred}[i, j, x]$
3. if $y \geq x$ and $C[i, k, y] \wedge C[k + 1, j, y - x]$ then
4. output “(”, $\text{OUTPUT-EXPR}(i, k, y)$, output “) $\ominus$ (”,
 $\text{OUTPUT-EXPR}(k + 1, j, y - x)$, output “)”
5. else
6. output “(”, $\text{OUTPUT-EXPR}(i, k, y)$, output “) $\ominus$ (”,
 $\text{OUTPUT-EXPR}(k + 1, j, y + x)$, output “)”

We initially call $\text{OUTPUT-EXPR}(1, n, b)$.

Analysis. In the first part of the pseudocode (to decide yes/no): Lines 6–8 take $O(nU)$ time (as we try all k and y). Thus, lines 3–8 take $O(n^2U \cdot nU) = O(n^3U^2)$. Lines 1–2 take $O(nU)$ time. Total time is $O(n^3U^2)$.

In the pseudocode to output the expression: The run time is proportional to the length of the output expression, which is $O(n)$. So, the first part dominates.

Final time bound: $O(n^3U^2)$.

Space bound: $O(n^2U)$ (to store the tables C , k-pred , and y-pred).

4. *Definition of subproblems.* Create an extra vertex v_0 and set the weight of (v_0, v_i) to 0 for each $i \in \{1, \dots, n\}$.

For $i \in \{1, \dots, n+1\}$ and $j_1, j_2 \in \{0, \dots, i-1\}$, let $f(i, j_1, j_2)$ be the maximum weight of two paths which start at v_{j_1} and v_{j_2} and afterwards use only vertices from the suffix $\{v_i, \dots, v_n\}$, with each vertex in $\{v_i, \dots, v_n\}$ used at most once.

The answer we want is $f(1, 0, 0)$.

Recursive formula. Base case: $f(n+1, j_1, j_2) = 0$ for all j_1, j_2 .

For $i \leq n$,

$$f(i, j_1, j_2) = \max \begin{cases} f(i+1, i, j_2) + w(v_{j_1}, v_i) \\ f(i+1, j_1, i) + w(v_{j_2}, v_i) \\ f(i+1, j_1, j_2) \end{cases}$$

Brief explanation: the 3 cases which we take max over corresponds to (i) using the first path to cover v_i (which adds weight $w(v_{j_1}, v_i)$ and resets j_1 to i), (ii) using the second path to cover v_i (which adds weight $w(v_{j_2}, v_i)$ and resets j_2 to i), and (iii) not covering v_i .

Evaluation order. Decreasing order of i .

Analysis. There are $O(n^3)$ table entries, each of which takes $O(1)$ time. Total time: $O(n^3)$.

Space: $O(n^3)$ naively.

Since we only want the optimal value, we can reduce space to $O(n^2)$ by remembering two rows of the table for f (i.e., $f(i, \cdot, \cdot)$ and $f(i+1, \cdot, \cdot)$).

[*Alternative solution.* There is a different DP solution using a 2-dimensional table instead of 3. The idea is to define the following subproblems: let $f(j_1, j_2)$ be the maximum weight

of two paths which start at v_{j_1} and v_{j_2} and afterwards use only vertices from the suffix $\{v_{\max\{j_1, j_2\}+1}, \dots, v_n\}$, with each vertex used at most once. Then the recursive formula becomes

$$f(j_1, j_2) = \max \begin{cases} \max_{j > \max\{j_1, j_2\}} (f(j, j_2) + w(v_{j_1}, v_j)) \\ \max_{j > \max\{j_1, j_2\}} (f(j_1, j) + w(v_{j_2}, v_j)) \end{cases}$$

(I will omit details such as base cases.) Although the number of table entries is reduced to $O(n^2)$, the time per entry increases to $O(n)$. The total run time is still $O(n^3)$.]